

Explaining State-Merging Techniques Through MIR Dumps

Yaroslav Riabtsev

Communication and Distributed Systems

RWTH Aachen University

Aachen, Germany

yaroslav.riabtsev@rwth-aachen.de

Abstract—This article explains state merging through inspectable MIR examples and dumps. Instead of evaluating a full symbolic executor by coverage, bugs found, generated tests, or solver runtime, it follows the intermediate structures that make merging decisions understandable: path splits and joins, conditional symbolic values, dependency sets, future-relevant variables, and compact summaries of repeated or regional control flow.

The paper starts from dynamic symbolic execution and standard state merging, where merging reduces the number of states but moves complexity into disjunctions and *ite*-expressions. It then uses small MIR examples to explain Query Count Estimation, hot variables, state similarity, Dynamic State Merging, Veritest-style region summarization, and indexed loop summaries for regular path families.

The central theme is the same throughout: a merge is useful when the formula complexity it introduces remains manageable. The MIR dumps make this trade-off visible by showing which path-dependent values reach future conditions, which variables become important for later queries, and which CFG or loop structures admit a compact summary.

Index Terms—symbolic execution, dynamic symbolic execution, state merging, query count estimation, dynamic state merging, Veritest, static symbolic execution, MIR, control-flow graph, path explosion

I. INTRODUCTION

KLEE is one of the most widely known practical systems for dynamic symbolic execution. It takes programs represented in LLVM IR, executes them over symbolic inputs, and maintains symbolic states corresponding to different possible executions of the program. This execution model is useful in automated test generation, coverage-oriented exploration, bug finding, and reachability analysis. For this reason, KLEE provides a convenient technical reference point for discussing more specialized symbolic-analysis techniques [1].

Several techniques considered in this article can be understood as extensions, modifications, or close relatives of a KLEE-like execution model. They preserve the same general view of a program as executable intermediate code with explicit control flow, but change how symbolic states, path predicates, program regions, or repeated path structures are represented and manipulated. The article therefore looks inside these techniques rather than treating them as black boxes, focusing on the local mechanics: which data an algorithm needs, which structures appear in the CFG, which expressions

are constructed, which heuristics guide decisions, and which intermediate artifacts determine these steps.

Throughout the article, a symbolic state is written as

$$s = (\ell, \pi, \mu).$$

Here, ℓ is the current program location: an instruction, a basic block, or a point in the CFG. The component π is the path condition, i.e. the constraint accumulated along the current path. The component μ is the symbolic store: a mapping from program variables to symbolic expressions. Accordingly, $\mu(x)$ denotes the current symbolic value of variable x . Different papers use different notation for this conceptual triple, such as location/path condition/store, instruction counter/path predicate/symbolic memory, or variants like (ℓ, Π, Δ) . This article uses (ℓ, π, μ) consistently.

KLEE operates on LLVM IR, but the experiments in this article use a smaller MIR-like representation. The representation contains only the constructs needed to demonstrate the techniques under discussion: parameters, assignments, arithmetic operations, conditional jumps, unconditional jumps, and returns. This level is sufficient to expose basic blocks, CFGs, join points, branch conditions, simple loops, variable dependencies, and candidate summaries.

The examples and tables were produced using an existing MIR analysis project reused as a lightweight playground [2]. The MIR experiments inspect static control-flow and dependency information; they do not perform path-feasibility solving or test generation. The article itself is self-contained: the dumps provide reproducibility artifacts, but the relevant listings, equations, and result tables are included in the text.

II. MOTIVATION AND BASELINE

Symbolic execution occupies a useful position between concrete execution and static reasoning about program behavior. In a concrete run, a program receives specific input values and follows one specific path. In symbolic execution, some inputs are replaced with symbolic values, and program operations construct expressions over those symbols. As a result, a single analysis state can represent not just one execution, but a family of executions satisfying a particular condition.

The practical motivation for such systems is usually described through the outputs of the analysis. If the condition

for following a path is satisfiable, a solver can construct a concrete input that makes the original program follow the same path. This makes symbolic execution useful for automatic test generation, line and branch coverage improvement, bug finding, reachability checking, and reproducible bug reports. This connection to concrete inputs is one of the reasons symbolic execution remains attractive: an analysis result can often be turned into an actual program run.

Another important layer is the SMT-solving layer. In practical symbolic execution systems, path conditions and symbolic expressions over inputs are translated into logical formulas. A solver is then used to check path feasibility, discard impossible branches, and construct models for concrete inputs. This layer accounts for a significant part of the practical cost of the analysis: even when the number of explored paths is moderate, the formulas themselves may become difficult for the solver.

This article focuses on the algorithmic foundations that support both of these aspects. Test generation, coverage, verification, and bug reporting provide the external motivation rather than experimental metrics. Instead, the article studies the intermediate structures on which such systems rely: symbolic states, path conditions, symbolic stores, control-flow graphs, join points, conditional expressions, variable dependencies, and local indicators of future complexity.

This level is especially useful for discussing state-merging techniques. These techniques are not reducible to a single formula or a single solver call. They use information from different parts of the analysis: where join points are located, which values differ across paths, whether those values may appear in future conditions, whether several paths can be represented by a regional summary, and whether a repeated pattern is present in a loop. Understanding the algorithms therefore requires looking at the intermediate artifacts on which their decisions are based.

A. Dynamic Symbolic Execution

KLEE is a path-based symbolic execution engine that dynamically interprets LLVM IR over symbolic states. A program is executed step by step, but input values may be symbolic rather than concrete. Operations over symbolic values construct expressions, and conditional branches add constraints to the path condition of the current state.

Consider the example in Listing 1:

```
1 int count_matches(int i, int n, int target) {
2     int matches = 0;
3
4     while (i < n) {
5         if (target == i) {
6             matches = matches + 1;
7         } else {
8             matches = matches + 0;
9         }
10
11         i = i + 1;
12     }
13
14     return matches;
15 }
```

Listing 1. C loop with repeated symbolic branch.

Assume that i , n , and $target$ are symbolic inputs. At line 4, the executor has to determine whether the loop continues. At line 5, it has to choose one of the two updates of `matches`.

When a branch condition depends on symbolic data, KLEE performs a fork: the current state is cloned, and the analysis continues with two states. One state receives the constraint for the true branch, and the other receives the constraint for the false branch. For example, entering the loop and taking the true branch of the inner condition gives the path condition $i < n \wedge target = i$. Taking the false branch instead produces another state with the condition $i < n \wedge \neg(target = i)$.

After the update $i = i + 1$, control returns to the loop condition, and the same branch structure repeats for the next value of i . If the loop is unrolled for k iterations and both continuations of the inner branch are kept as syntactic variants, then the inner branch alone defines up to 2^k branch histories.

For this concrete scalar example, an SMT-backed executor may prune many combinations, because $target = i$ can be true for at most one loop iteration. The MIR-level experiments keep the syntactic path family induced by the CFG; this gives the structural path family before feasibility pruning.

KLEE does not add such branches blindly. For each side of a conditional branch, it checks the satisfiability of the corresponding path condition. If $\pi \wedge c$ is unsatisfiable, the true branch is discarded. If $\pi \wedge \neg c$ is unsatisfiable, the false branch is discarded. The SMT solver therefore partially limits the growth of states: a syntactically available branch may be infeasible and will not be explored further.

The experiments use the MIR-like version of the same example, shown in Listing 2.

```
1 param i
2 param n
3 param target
4 matches <-- #0
5 Lloop: iffFalse i < n goto Lend
6 iffFalse target = i goto Lskip
7 matches <-- +, matches, #1
8 goto Lnext
9 Lskip: matches <-- +, matches, #0
10 Lnext: i <-- +, i, #1
11 goto Lloop
12 Lend: return matches
```

Listing 2. MIR form of the repeated-branch loop.

The MIR form makes basic blocks and transitions explicit: the loop condition becomes a conditional jump from `Lloop`, the inner condition becomes a second conditional jump, and the update of i returns control to the loop header. This structure is enough to inspect CFG-level path growth: if the CFG contains two syntactic continuations, both are visible as potential variants in the structural experiment.

As a result, the experiment exposes path growth in its pure structural form. Each binary symbolic branch doubles the number of potential histories, and a repeated branch inside a loop produces an exponential family of paths. This is path explosion.

B. Standard State Merging

Path explosion arises because dynamic symbolic execution keeps different branch histories as different states. If paths

diverge permanently, this is natural: they describe genuinely different continuations of the execution. However, in many programs, branches later reconverge at the same program location. In that case, it becomes possible to continue the analysis not from several independent states, but from one merged state.

The idea of standard state merging is to combine several states located at the same program point [3], [4]. For simplicity, the following equations assume that the two states are merge-compatible: they are located at the same program point and their stores are defined over the same variables relevant after the merge.

Let two such states be

$$s_1 = (\ell, \pi_1, \mu_1),$$

$$s_2 = (\ell, \pi_2, \mu_2).$$

Both states are at the same location ℓ , but they reached it through different paths. The merged state must preserve information about both paths. Therefore, its path condition becomes a disjunction:

$$\pi = \pi_1 \vee \pi_2. \quad (1)$$

Variable values must also depend on the path from which the execution arrived. For this purpose, state merging uses conditional expressions of the form *ite*, meaning if-then-else. For a variable x , the merged store can be written as

$$\mu(x) = ite(\pi_1, \mu_1(x), \mu_2(x)). \quad (2)$$

That is, if the path condition of the first state holds, the value is taken from $\mu_1(x)$; otherwise, it is taken from $\mu_2(x)$. In general, merging reduces the number of states, but moves path differences into formulas.

In the example from the previous subsection, this can be seen on the variable `matches`. In Listing 1, lines 5–9 select one of two updates: line 6 increments `matches`, while line 8 leaves it unchanged.

After these branches, control reaches the same point before the update of `i`. Instead of keeping two separate states, the value of `matches` can be represented by one conditional expression:

$$matches' = ite(target = i, matches + 1, matches + 0).$$

This representation follows the store shape in Equation (2) and remains precise: both branches are still represented in the formula. At the same time, the number of active states decreases, because the later analysis can continue from one merged state.

However, standard state merging does not remove complexity; it changes its form. Before merging, the complexity is visible as the number of states and paths. After merging, it appears inside the path condition and symbolic store: disjunctions and *ite*-expressions are introduced. If such conditional

values are later used in new branch conditions, solver queries may become harder.

Thus, state merging is a trade-off. It can reduce path explosion at the level of the number of states, but it may increase formula complexity. The following parts of the article examine this trade-off: when merging looks useful, when it becomes risky, and which intermediate data helps make that decision.

a) *ITE-cost shape: conditional values in later branches:*

To make the cost of state merging visible, it is useful to look both at the fact that states are merged and at where the introduced *ite*-expression flows afterwards. If a conditional value is not used in later branch conditions, the merge may look relatively safe. If the value reaches a future branch condition, then the complexity introduced by merging is pushed into a later query.

Consider the MIR fragment in Listing 3:

```

1 param x
2 param y
3 ifTrue x < #0 goto Lneg
4 a <-- +, x, #5
5 goto Ljoin
6 Lneg: neg <-- -, #0, x
7 a <-- +, neg, #1
8 Ljoin: t <-- +, a, y
9 ifTrue t > #0 goto Lhot
10 r <-- +, y, #1
11 goto Lend
12 Lhot: r <-- +, t, #1
13 Lend: return r

```

Listing 3. MIR branch with a join and a later hot branch.

The first branch splits execution on the condition $x < 0$.

At `Ljoin`, the variable `a` has different expressions depending on the selected branch:

$$a = \begin{cases} 1 - x, & x < 0, \\ x + 5, & \neg(x < 0). \end{cases}$$

A state-merge-style representation of this value can be written as $a = ite(x < 0, 1 - x, x + 5)$. Immediately after the join block, the program computes $t = a + y$, and then checks the condition $t > 0$. The incoming path conditions are mutually exclusive, and the conditional value is interpreted under their disjunction.

Substituting the shallow *ite*-representation of `a` into this branch condition gives

$$ite(x < 0, 1 - x, x + 5) + y > 0.$$

This is the important shape for the later analysis: the *ite*-expression does not simply sit in the symbolic store; it reaches a future branch condition. That makes the merge candidate interesting and potentially risky.

For the merge site `Ljoin` and the variable `a`, the experiment reconstructs the sequence

$$a = ite(x < 0, 1 - x, x + 5), \quad t = a + y,$$

and then observes that the branch $t > 0$ depends on this conditional value. The formula is small, but it already demonstrates the mechanism: merging reduces the number of states

while introducing a conditional value that enters the next branch condition.

The final value r at `Lend` gives the contrasting case. An *ite*-representation can be constructed for that value as well, but there is no later branch condition after that point. This candidate is less dangerous with respect to future branch queries: the conditional value exists, but it is not pushed into another condition.

The important distinction is future use. The local existence of an *ite*-expression is not enough to characterize merge cost; the relevant question is whether the path-dependent value reaches a later condition.

b) Hash-consed expressions: Besides path explosion, state merging also creates pressure on memory usage and on the cost of traversing symbolic expressions. After a merge, the symbolic store contains *ite*-nodes. After SSA/phi-style representation and variable versioning, the same subexpressions may appear in several merge candidates, future-branch substitutions, and $\hat{Q}_{\text{add}}$ -dependencies. If every step constructs a fresh expression tree, identical fragments are duplicated even when they represent the same object.

Here, hash-consing is background from prior symbolic-execution work: it shares expression structure by assigning each expression node a structural key consisting of the node kind, its local payload, and the identifiers of its children [5]:

$$\text{key}(e) = (\text{kind}(e), \text{payload}(e), \text{id}(e_1), \dots, \text{id}(e_n)).$$

The payload may contain an operator name, a constant, a variable name, a variable version, or the information that the node is an *ite*. Before a new node is created, the table of already constructed expressions is queried by this key. If such a node already exists, its identifier is reused; otherwise, a new node is allocated. As a result, symbolic expressions are stored as a shared DAG rather than as a collection of independent trees.

For merged symbolic expressions, hash-consing reduces duplication and repeated traversal without changing the meaning of the formulas. Counts of *ite*-nodes, expression size, phi-related merge candidates, versioned dependencies, and future-branch substitution can all be computed over one shared representation instead of over duplicated trees.

III. QUERY COUNT ESTIMATION AND DYNAMIC STATE MERGING

A. Query Count Estimation

Efficient State Merging introduces Query Count Estimation as a way to estimate whether the expected benefit of merging outweighs the possible increase in the complexity of future solver queries. In the source paper, Q_t estimates the expected total number of future solver queries, Q_{add} concerns additional queries caused when a value that differs between states becomes symbolic after merging, and Q_{ite} concerns future queries affected by conditional expressions introduced by the merge [3]. Variable-specific estimates then support the identification of hot variables.

The *ite*-flow example in Section II-B illustrates the Q_{ite} side of this cost: a conditional value introduced at a join reaches a later branch condition. Appendix B manually applies the source-paper q recurrence to the MIR graph and produces the oracle Q_t and Q_{add} values used in the comparison below.

The MIR experiment uses a separate hatted notation for the quantities computed automatically from the CFG and recovered dependencies. For each block ℓ , $\hat{Q}_t(\ell)$ is a CFG-derived estimate of future conditional-branch pressure. For a variable v , $\hat{Q}_{\text{add}}(\ell, v)$ is the part of that pressure attributed to v by the selected dependency approximation. The local dependency indicator $C(\ell, v)$ is non-zero when the selected dependency approximation decides that the branch condition at ℓ depends on v . The experimental hot set is therefore

$$\widehat{\text{Hot}}(\ell) = \{v \mid \hat{Q}_{\text{add}}(\ell, v) > \alpha \cdot \hat{Q}_t(\ell)\}.$$

Conceptually, β controls how strongly a later branch influences earlier locations. A larger value propagates more of the future pressure backward through the CFG. The threshold α controls how selective the experimental hot-variable set is: small values mark more variables as hot, while larger values keep only variables whose $\hat{Q}_{\text{add}}$ is a substantial fraction of $\hat{Q}_t$.

In the default configuration, $\alpha = 0.35$ and $\beta = 0.50$. For an acyclic branch graph, the MIR recurrence can be summarized as

$$\hat{Q}_t(\ell) = \begin{cases} 1 + \beta \sum_{s \in \text{succ}(\ell)} \hat{Q}_t(s), & \ell \text{ is conditional,} \\ \sum_{s \in \text{succ}(\ell)} \hat{Q}_t(s), & \text{otherwise,} \end{cases}$$

and, for conditional blocks,

$$\hat{Q}_{\text{add}}(\ell, v) = C(\ell, v) + \beta \sum_{s \in \text{succ}(\ell)} \hat{Q}_{\text{add}}(s, v).$$

For non-conditional blocks, the local $C(\ell, v)$ term is omitted and only successor pressure is propagated.

The same fixed-point computation gives both hatted quantities; only the local cost function changes.

```

algorithm SolveFutureCost(CFG, branch_successors, local_cost
):
  q[block] = 0 for every block

  repeat until q changes by less than tolerance:
    next_q = empty vector

    for block in reverse block order:
      if block is a conditional branch:
        next_q[block] =
          local_cost(block)
          + beta * sum(q[s] for s in branch
            successors)
      else if block has one successor:
        next_q[block] = local_cost(block) + q[
          successor]
      else:
        next_q[block] =
          local_cost(block)
          + beta * sum(q[s] for s in successors)

    q = next_q

  return q

```

Listing 4. Fixed-point propagation used for the hatted MIR estimates.

In the MIR example below, future conditional branches are used as the query-pressure signal, while variable dependencies determine which source values contribute to that pressure. For this acyclic graph, the automatic $\hat{Q}_t$ recurrence has the same control-flow shape as the source-paper Q_t oracle; the principal experimental approximation is the recovered dependence relation used for $\hat{Q}_{\text{add}}$.

B. Dependency Approximation Experiment

The MIR graph in Listing 3 contains the dependency chain that drives the QCE example. The later branch is written as $t > \#0$; t is computed as $a + y$; and a is defined on both incoming paths from the earlier branch. Therefore, the source-level dependency of the later branch is x, y , even though the branch expression contains only t .

The Appendix B oracle uses the source-paper two-location relation $(\ell, v)\mathcal{C}(\ell', e)$: the value of v at location ℓ may affect the expression e evaluated at the later location ℓ' . The four automatic dependency modes approximate this relation from MIR syntax and CFG structure; the manual oracle is not a fifth implemented mode.

The modes expose progressively deeper representations of the same dependency chain rather than competing in an identical variable namespace. Syntactic mode retains the branch temporary t ; local expansion reaches a and y ; CFG propagation resolves a to the source variables x and y ; and versioned propagation records the same sources with definition versions.

The syntactic approximation reads variables directly from branch expressions. For $t > \#0$, it reports only t . This is a useful minimal baseline because it locates query pressure cheaply: the analysis can already see that block D contains a future branch. What it cannot explain is why that branch is relevant to a merge performed earlier. The temporary t is still a surface variable, not the source of the path-dependent value.

The local approximation expands assignments inside the current block. Since $t = a + y$ appears immediately before the branch, local expansion reports a, y . This removes the temporary t and exposes that a and y are the values consumed by the later condition. The important remaining gap is conceptual: the analysis knows that a matters, but not why a differs between the incoming paths.

The CFG approximation carries dependency sets along CFG edges and unions them at joins. It resolves a through both predecessor definitions and therefore recovers x, y , which matches the oracle source variables for this example. This is the point where the earlier branch on x becomes connected to the later branch on t . The union at joins is path-insensitive, but for this graph it is precise enough to explain the merge candidate.

The versioned approximation associates dependencies with source versions, giving x_0, y_0 . Its numerical result is the same here because x and y have one relevant source definition.

The representation matters in examples with redefinitions: it distinguishes a source value that reaches a future branch from a later assignment to a variable with the same name. The comparison is summarized in Table I.

C. QCE Results

For the default configuration ($\alpha = 0.35$, $\beta = 0.50$), Table II places the automatic MIR estimates beside the manually computed source-paper oracle. Table II(a) aggregates the fixed-point result for each dependency mode. *Vars* counts variables with non-zero contribution, *Blocks* counts blocks where future dependency information is visible, *Total $\hat{Q}_{\text{add}}$* sums the variable-attributed pressure, and *Hot refs* counts block-variable references that pass the $\bar{H}$ ot threshold. The displayed hatted quantities, aggregate masses, and hot-reference counts come from the automatic MIR analysis. Table II(b) contains the manual oracle. *Oracle L_1* is computed separately by projecting those estimates onto the source-variable domain of the manual oracle in Appendix B:

$$L_{1,\text{oracle}}(m) = \sum_{\ell} \sum_{v \in V_{\text{oracle}}} \left| \hat{Q}_{\text{add}}^m(\ell, v) - Q_{\text{add}}(\ell, v) \right|.$$

For this example, $V_{\text{oracle}} = \{x, y\}$. Intermediate variables reported by shallower modes, such as t or a , are not treated as false positives; their effect appears as source-level attribution that has not yet been recovered. For the versioned mode, source-version names such as x_0 and y_0 are normalized to their corresponding source variables before computing the distance. This article-level metric is distinct from the dumper’s internal $L_1\Delta$ vs CFG summary, which compares automatic modes against CFG mode inside their reported variable spaces.

The main result is not the exact total mass but the recovered variable identity. Syntactic mode retains only surface variables. Local mode improves data-flow recovery inside the block, but still attributes relevance to the intermediate join value a . CFG mode matches the oracle for this example: it recovers the source variables x, y , the per-block Q_{add} values, and the oracle hot-variable set. Versioned mode has the same numerical agreement after normalizing x_0, y_0 to x, y . Syntactic and local modes preserve part of the dependency at intermediate program variables and therefore recover only part of the source-level attribution measured by the oracle. Increasing β gives later branches more influence over earlier blocks, but it does not change the qualitative comparison between dependency approximations.

D. Merge Decision

The hatted estimates are then used for a lightweight merge-risk question:

If a join merges values from multiple predecessors, do those merged values flow into future hot branch conditions?

For each join candidate, the experiment identifies the join, recovers values arriving from its predecessors, determines their source dependencies, compares those dependencies with the

experimental future-hot set $\widehat{\text{Hot}}(\ell)$, and interprets the overlap. The comparison uses the CFG dependency mode as the default source of future-hot variables. The two relevant rows are shown in Table III.

There are two different sets involved in this comparison. The future-hot set describes variables that can affect later branch pressure after the join. The merged-source set describes variables behind values that actually differ between the incoming states. A merge is risky when these sets overlap. A future-hot variable that is not part of the path-specific difference is still important to the later query, but it is not the reason this particular merge introduces path-dependent structure.

The labels in Table III are experimental labels for this overlap test. The relevant comparison is

$$\text{DepsOfDifferingValue} \cap \widehat{\text{Hot}}(\ell).$$

A non-empty partial overlap is reported as *mixed*; a candidate after which $\bar{Q}_t = 0$ is reported as *no-future-query*.

The central row is `D/Ljoin/a`: branch-specific definitions of a are reconstructed as $a = \text{ite}(x < 0, 1 - x, x + 5)$, the block computes $t = a + y$, and the following branch checks $t > 0$.

The merge of a at `Ljoin` has consequences beyond local SSA. The merged value can influence the later branch through t . Both incoming definitions of a depend on x , so the path-specific difference being merged is traced back to x . The later branch depends on x and y ; y is future-hot because it participates in $t = a + y$, but it is not the path-specific difference introduced by the merge. The relevant overlap is therefore x , which produces the *mixed* decision. The final-value merge at `G/exit` is different: r may have different incoming expressions, but no later condition consumes it, so no future branch pressure remains and the row is labeled *no-future-query*.

The result supports the central state-merging point: a merge is potentially risky not just because it creates an *ite*, but because the differing value remains relevant to future branch conditions. This type of relevance information could complement runtime merge heuristics without requiring the full query model to be reconstructed statically.

A one-off call to `z3.simplify` was used to check the displayed form of the reconstructed expression; the *ite* structure remained unchanged [6].

E. State Similarity and Dynamic State Merging

Dynamic State Merging uses QCE to define a similarity relation between states. Two states at compatible program locations need not have identical symbolic stores. If they differ only in variables that are cold with respect to future queries, merging them is less likely to increase later formula cost. If they differ in hot variables, merging is less attractive because those differences are likely to appear in future branch conditions or other solver queries.

Thus, QCE supplies the relevance dimensions for comparing symbolic stores. The state similarity question is not simply whether two stores are equal, but whether they differ

in variables that matter after the merge point. The `Ljoin` example illustrates this relation: the path-dependent value of a is relevant because it flows into $t > \#0$; a value that differs only after the last branch is less important for future query complexity.

A compact way to express the similarity test is

$$\begin{aligned} s_1 \sim_\ell s_2 &\iff \forall v \in \text{Hot}(\ell) : \\ &\mu_1(v) = \mu_2(v) \vee \text{Sym}_I(\mu_1(v)) \\ &\vee \text{Sym}_I(\mu_2(v)). \end{aligned}$$

Here, $\text{Sym}_I(e)$ means that expression e is already symbolic with respect to the symbolic input set I . Equality on hot variables is the clean case. A difference may also be acceptable when at least one side is already symbolic, because merging then does not newly turn that variable from a concrete value into a symbolic value. Cold variables do not disappear from the program state, but they do not dominate the similarity decision.

The MIR dependency experiment supplies the relevance part of this relation: the hatted future-hot set indicates which variables should be compared.

DSM then addresses the scheduling side of the same problem. In ordinary dynamic symbolic execution, active states are stored in a worklist, and `pickNext` selects which state should be continued next. This driving heuristic may optimize for depth, coverage, random-path behavior, or another exploration criterion [3]. States that would be profitable to merge may not naturally reach the same join point under that order.

DSM preserves the underlying driving heuristic but temporarily gives priority to forwarding candidates. Predecessor history records recent states within a bounded distance. When an active state is similar to a recent predecessor of another active state, the algorithm treats this as a potential merge opportunity and places suitable states into a forwarding set. These candidates are not chosen because the normal search heuristic would necessarily prefer them globally; they are chosen because the history relation suggests that they may soon reach compatible locations. Forwarding ends naturally when states merge, when the opportunity no longer holds, or when the similarity condition stops applying. After that, exploration returns to the underlying search strategy.

The dependency example illustrates the relevance information used by this similarity relation, while DSM describes how such information is used during exploration.

IV. STATIC SYMBOLIC EXECUTION AND VERITESTING

Dynamic symbolic execution explores a program path by path: each state corresponds to one branch history. Static Symbolic Execution uses a different analysis unit. Instead of maintaining a separate state for each path, SSE tries to construct a formula for an entire program fragment. For a small acyclic region, this can be written in a simplified form as

$$\text{Summary}(R) = \bigvee_{p \in \text{paths}(R)} PC_p \wedge \text{Update}_p. \quad (3)$$

Here, PC_p describes the conditions required to follow path p , and $Update_p$ describes the corresponding symbolic-store update. This makes SSE a natural candidate for compressing local branch diamonds: several paths inside a region can be represented by one summary structure in the form of Equation (3).

Veritesting uses this idea as part of dynamic analysis. At a candidate point, the system recovers a local CFG from the current execution state, reduces that CFG, identifies transition points, and applies SSE to the reduced acyclic graph [5]. Branch contexts and symbolic stores are propagated inside the static region; at joins they are merged into a summary rather than materialized as one state per path. At transition points, DSE states are recreated and ordinary path exploration resumes.

Transition points are the boundary between the two modes. They appear at exits of the static region, at unresolved jumps, at calls or system interactions that require the dynamic engine, and at loop-related cases handled through unrolling, back-edge redirection, or an Incomplete Loop transition. This alternation is what makes Veritesting different from using DSE or SSE alone: static summaries are used locally, while the dynamic engine remains responsible for the rest of the execution.

Java Ranger gives method calls a more central role because Java programs often contain many small methods, and dynamically dispatched calls depend on runtime receiver types [7]. Inheritance and object-oriented dispatch make method boundaries frequent and semantically important, so treating every method call as an unconditional boundary would fragment multi-path regions. Java Ranger therefore introduces Method Regions and Dynamic Method Region Inlining: after runtime type information identifies the applicable target, a method region can be instantiated and inlined into the regional summary. Exceptional or otherwise unsuitable behaviors may remain separate Single-Path Cases.

The MIR examples below use a narrower direct acyclic-region preview. The preview asks:

Does this branch-shaped part of the CFG look like a small acyclic region that could be summarized?

The preview uses conditional entries, common exits or immediate postdominators, the node set between them, back-edge presence, and directly acyclic path sketches. It isolates one structural ingredient of Veritesting: recognizing a region whose shape admits a summary.

The direct acyclic-region preview is shown in Listing 5.

```

algorithm DirectAcyclicRegionPreview(CFG, branches, ipdom):
  for entry in branches:
    exit = ipdom(entry)
    region = nodes reachable before exit

    if region contains a cycle:
      row(entry, exit, region, status = "not directly
acyclic")
    else:
      paths = bounded acyclic paths(entry, exit)
      row(entry, exit, region, paths)

```

Listing 5. Direct acyclic-region preview.

The first test MIR graph contains two acyclic diamond structures (Listing 6).

```

1 param x
2 param y
3 ifTrue x > #0 goto Lpos
4 neg <-- -, #0, x
5 a <-- +, neg, y
6 goto Ljoin
7 Lpos: a <-- +, x, y
8 Ljoin: ifFalse y == #0 goto Lscale
9 r <-- +, a, #1
10 goto Lend
11 Lscale: r <-- *, a, y
12 Lend: return r

```

Listing 6. Acyclic diamond MIR input.

The preview identifies two regions that can be represented as compact acyclic path sketches, as shown in Table IV.

Both regions have two syntactic branches and one common exit. This is the case where a Veritesting-style approach is natural: instead of keeping two separate DSE continuations, the local region can be viewed through a summary form.

The second test reuses the loop example from earlier sections (Listing 2). In that listing, line 5 is the loop header, line 6 is the inner branch, and lines 10–11 form the latch and back edge.

Here there are two different candidate regions. One starts at the loop header and contains a back edge. The other is the inner branch diamond inside the loop body. The direct preview distinguishes these cases in Table IV.

This behavior is important: the loop-shaped candidate is not directly acyclic in the original CFG because it contains the back edge. A full Veritesting treatment would require CFG reduction, unrolling, or a transition to dynamic handling. The inner branch inside the loop is still a directly acyclic diamond region and can be summarized separately.

A manual check gives the same classification; the same table records the expected structural reason for each outcome.

Thus, the MIR examples illustrate a region-shape preview connected to Veritesting. Directly acyclic branch diamonds are classified as summary-shaped candidates, while a selected fragment with a back edge is marked as not directly acyclic. The inner branch of the loop is still a directly acyclic region, which shows that the preview depends on the selected CFG fragment, rather than on whether the surrounding function contains a loop.

V. PATTERN-BASED AND QUANTIFIED STATE MERGING

The previous section considered region-level summarization for acyclic CFG fragments. Loops introduce a different kind of structure: not just several branches inside one diamond region, but a repeated family of similar paths. If the loop body contains a symbolic branch, ordinary DSE can represent different combinations of branch outcomes across iterations. Standard state merging can merge individual join points, but with many repetitions the resulting nested *ite*-expressions may still grow.

Quantified merging targets groups of execution-tree states whose histories do not differ arbitrarily, but instead follow a

regular pattern. The typical source-paper shape is a family of leaves whose histories can be written as

$$h(\pi(\text{leaf})) = \omega_1 \omega_2^k \omega_3.$$

The prefix ω_1 reaches the repeating fragment, ω_2 is the loop-body pattern repeated k times, and ω_3 exits the fragment. Compatible leaf states can then be partitioned into regular merging groups. For such a group, repeated path constraints are encoded with a bounded quantified schema whose k ranges over repetition counts represented by the selected compatible states rather than every arbitrary unseen count; in the MIR preview below, k is used as an explanatory loop-iteration index. Store values are synthesized as functions of the repetition count k where possible. Ordinary *ite*-based merging remains available for variables whose terms cannot be synthesized, and for groups where the regular pattern is not applicable [8].

The MIR experiment uses a CFG-level indexed-family preview. It approaches the same regularity phenomenon from the CFG side: instead of constructing an execution-tree merging group, it searches the loop for the structural elements that would generate an indexed family: loop guard, induction update, inner branch, guarded accumulator update, and repeat edge.

The loop/accumulator MIR example reuses Listing 2. The pattern experiment uses the loop guard on line 5, the inner branch on line 6, the two *matches* updates on lines 7 and 9, and the induction/back-edge shape on lines 10–11.

The corresponding source-level loop shape is the body of Listing 1, especially lines 2–12.

The repeated inner branch is *target = i*.

If the loop starts at i_0 and exits at n , the preview can write the following compact indexed sketch:

$$\text{matches}_{\text{out}} = \text{matches}_{\text{in}} + \sum_{k=i_0}^{n-1} \text{ite}(\text{target} = k, 1, 0). \quad (4)$$

Equation (4) shows the regularity exposed by the preview: the same conditional update is repeated over the loop index. The summation is an explanatory sketch of the indexed family for this MIR loop. The loop guard defines the range of k , the induction update defines how k advances, the inner branch defines the repeated condition, and the accumulator update supplies the per-iteration contribution.

The compact recognition sketch in Listing 7 checks for the CFG components needed by this indexed view.

```

algorithm CfgIndexedFamilyPreview(CFG, branches, assignments
):
  find a loop back edge latch -> header
  recover nodes between header and latch

  guard = branch condition at header
  induction = linear update of the loop variable
  accumulator = inner branch with two constant updates

  if guard, induction, and accumulator are found:
    rewrite the branch condition with iteration variable
    k
    emit indexed sketch:

```

```

accumulator += ite(condition(k), true_delta,
false_delta)

```

Listing 7. CFG-level indexed-family preview.

The sketch looks for one back edge, recovers the loop body, and then identifies three components: a guard, a linear induction update, and a guarded accumulator. If all components are found, the result is an indexed sketch for one iteration and for the range of k .

For this MIR graph, the bounded node tree is shown in Table V.

From this tree, the pattern-inference and manual-check summary in Table VI is extracted:

The same result can be described as the small automaton in Listing 8.

```

S0 -- not (i < n) -----> ACCEPT-EXIT
S0 -- i < n -----> S1
S1 -- target = i / matches += 1 -----> S2
S1 -- not (target = i) / matches += 0 -----> S2
S2 -- i += 1; k += 1 -----> S0
S0 + S1 + S2 recognized -----> ACCEPT

```

Listing 8. Finite automaton view of the recognized loop pattern.

The automaton and the bounded node tree serve the same purpose: they show how a parameterized family of repeated histories is generated from a small structural pattern. The finite cases are loop exit, loop entry, true/false arm of the inner branch, latch update, and repeat. Once these cases are recognized, the repeated branch family can be described by one indexed schema.

A manual comparison with the target shape is included in the same table.

On the small loop example, the recovered guard, induction update, inner branch, and accumulator update lead to an indexed sketch of the form

$$\text{for } k \in [i_0, n): \quad \text{matches} += \text{ite}(\text{target} = k, 1, 0).$$

Recognizing this structure before formula construction may help select cases where quantified merging’s execution-tree pattern search is likely to find a regular group.

Automatic partial loop summarization is related, but it operates dynamically: it observes loop executions, matches induction-variable and guard behavior, predicts and confirms a symbolic iteration count, and emits partial pre-loop and post-loop conditions. Quantified merging instead compresses regular groups of symbolic states and path histories.

VI. RELATED WORK

KLEE is the main baseline for the execution model used in this paper: it executes LLVM IR over symbolic states, forks states at symbolic branches, checks path feasibility with an SMT solver, and explores the resulting state space [1]. SymCC is related as an alternative implementation strategy for symbolic execution, where instrumentation is compiled rather than interpreted [9].

Efficient State Merging is the closest reference for the QCE and DSM discussion [3]. In that work, Q_t , Q_{add} , and Dynamic

State Merging guide real merge decisions during symbolic execution. QCE supplies the future-relevance model used to reason about hot variables, while DSM explains how state similarity can guide exploration toward merge points. MultiSE is related through value summaries, which also represent multiple path values inside one symbolic representation [4].

Veritesting motivates the direct acyclic-region preview [5]. It combines dynamic symbolic execution with static symbolic execution over reduced local regions. Java Ranger extends a similar region-summary idea to Java with Method Regions, Multi-Path Regions, dynamic method-region inlining, Early>Returns Summarization, and exceptional Single-Path Cases [7]. The experiment in Section IV examines one region-shape question behind those approaches.

Quantified state merging motivates the CFG-level indexed-family preview [8]. The experiment in Section V examines whether a regular loop shape with a guard, induction update, inner branch, and guarded accumulator update can expose an indexed family.

Loop summarization and compositional dynamic test generation provide broader context for summarizing repeated or reusable behavior [10], [11]. Heap-oriented symbolic execution and modular heap summaries address richer memory structures than the scalar MIR examples used in this paper [12], [13]. CFM-SE is a concrete targeted control-flow melding transformation for reducing branching pressure [14]. Targeted program transformations for symbolic execution argue more broadly that such transformations should be first-class scalability components [15].

VII. CONCLUSION

State merging trades state count for formula complexity. The examples in this paper show why future relevance matters: a conditional value is most important when it flows into later branch conditions, and dependency precision determines which variables appear hot. In the MIR example, CFG-level propagation is enough to recover the relevant source variables, while purely syntactic or block-local approximations lose part of the explanation.

Hot variables connect QCE to Dynamic State Merging by defining the dimensions used for state similarity. Veritesting generalizes the merging idea from values at a join point to acyclic CFG regions, and quantified merging generalizes it again to regular loop-path families. Some of the required information resembles data already computed by control-flow and dependency analyses, but the main lesson is about state merging itself: merging is useful when the path differences that remain in formulas are controlled and future-relevant differences are handled carefully.

REFERENCES

- [1] C. Cadar, D. Dunbar, and D. Engler, "KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs," in *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation*, ser. OSDI '08. USENIX Association, 2008, pp. 209–224.
- [2] Y. Ryabtsev, "compilers-course," 2026. [Online]. Available: <https://github.com/yaryabtsev/compilers-course>
- [3] V. Kuznetsov, J. Kinder, S. Bucur, and G. Candea, "Efficient state merging in symbolic execution," in *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '12. New York, NY, USA: Association for Computing Machinery, Jun. 2012, pp. 193–204.
- [4] K. Sen, G. Necula, L. Gong, and W. Choi, "MultiSE: Multi-path symbolic execution using value summaries," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE '15. New York, NY, USA: Association for Computing Machinery, Aug. 2015, pp. 842–853.
- [5] T. Avgerinos, A. Rebert, S. K. Cha, and D. Brumley, "Enhancing symbolic execution with veritesting," in *Proceedings of the 36th International Conference on Software Engineering*, ser. ICSE '14. New York, NY, USA: Association for Computing Machinery, May 2014, pp. 1083–1094.
- [6] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Tools and Algorithms for the Construction and Analysis of Systems*, ser. Lecture Notes in Computer Science, vol. 4963. Berlin, Heidelberg: Springer, 2008, pp. 337–340.
- [7] V. Sharma, S. Hussein, M. W. Whalen, S. McCamant, and W. Visser, "Java Ranger: Statically summarizing regions for efficient symbolic execution of Java," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE '20. New York, NY, USA: Association for Computing Machinery, Nov. 2020, pp. 123–134.
- [8] D. Trubish, N. Rinetzy, S. Shoham, and V. Sharma, "State merging with quantifiers in symbolic execution," Aug. 2023. [Online]. Available: <https://arxiv.org/abs/2308.12068>
- [9] S. Poeplau and A. Francillon, "Symbolic execution with SymCC: Don't interpret, compile!" in *29th USENIX Security Symposium*, ser. USENIX Security '20. USENIX Association, 2020, pp. 181–198.
- [10] P. Godefroid and D. Luchaup, "Automatic partial loop summarization in dynamic test generation," in *Proceedings of the 20th International Symposium on Software Testing and Analysis*, ser. ISSTA '11. Association for Computing Machinery, 2011, pp. 23–33.
- [11] P. Godefroid, "Compositional dynamic test generation," in *Proceedings of the 34th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL '07. New York, NY, USA: Association for Computing Machinery, Jan. 2007.
- [12] J. Berdine, C. Calcagno, and P. W. O'Hearn, "Symbolic execution with separation logic," in *Programming Languages and Systems*, ser. Lecture Notes in Computer Science, vol. 3780. Berlin, Heidelberg: Springer, 2005, pp. 52–68.
- [13] I. Dillig, T. Dillig, A. Aiken, and M. Sagiv, "Precise and compact modular procedure summaries for heap manipulating programs," in *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '11. New York, NY, USA: Association for Computing Machinery, Jun. 2011, pp. 567–577.
- [14] C. Saumya, M. Hassan, R. Gangaraju, M. Kulkarni, and K. Sundararajah, "Taming the hydra: Targeted control-flow transformations for dynamic symbolic execution," *Proceedings of the ACM on Programming Languages*, vol. 10, no. OOPSLA1, Apr. 2026.
- [15] C. Cadar, "Targeted program transformations for symbolic execution," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE '15. New York, NY, USA: Association for Computing Machinery, Aug. 2015, pp. 906–909.

APPENDIX A

TABLES

TABLE I
DEPENDENCY SETS RECOVERED BY THE FOUR MIR DEPENDENCY APPROXIMATIONS.

Mode	Direct branch variable	Recovered dependency	Note
Syntactic	t	t	Reads variables directly present in the condition.
Local	t	a, y	Expands the assignment $t = a + y$.
CFG	t	x, y	Propagates dependencies through predecessor blocks.
Versioned	t	$x0, y0$	Records source definitions explicitly.

TABLE II
AUTOMATIC MIR ESTIMATES AND MANUAL SOURCE-PAPER QCE ORACLE.

(a) Automatic MIR estimates							(b) Manual source-paper QCE oracle				
Mode	Vars	Blocks	$\max \hat{Q}_t$	$\Sigma \hat{Q}_{\text{add}}$	Oracle L_1	Hot	Blocks	β	Q_t	$Q_{\text{add}}(x)$	$Q_{\text{add}}(y)$
Syntactic	2	2	2.00	7.00	10.00	7	Entry, A	0.50	2.00	2.00	1.00
Local	3	2	2.00	12.00	5.00	12	Entry, A	0.75	2.50	2.50	1.50
CFG	2	2	2.00	12.00	0.00	10	Entry, A	0.90	2.80	2.80	1.80
Versioned	2	2	2.00	12.00	0.00	10	B, C, D	any	1.00	1.00	1.00
							E, F, G, Exit	any	0.00	0.00	0.00

TABLE III
MERGE-CANDIDATE OVERLAP WITH EXPERIMENTAL FUTURE-HOT VARIABLES.

Join	Var	Source deps	Future hot	Overlap	Decision	Reason
D/Ljoin	a	x	x, y	x	mixed	a flows into $t = a + y$; the later branch uses t .
G/exit	r	x, y	none	none	no-future-query	No later branch pressure remains after this join.

TABLE IV
DIRECT ACYCLIC-REGION PREVIEW RESULTS.

Case	Entry-exit	Nodes	Observed outcome	Structural check
First diamond	$A \rightarrow D$	A, B, C	2 paths: $\neg(x > 0) \rightarrow D, x > 0 \rightarrow D$	Acyclic branch with common exit.
Second diamond	$D \rightarrow G$	D, E, F	2 paths: $y = 0 \rightarrow G, \neg(y = 0) \rightarrow G$	Acyclic branch with common exit.
Loop header	$B \rightarrow G$	B, C, D, E, F	Not directly acyclic; no path sketch.	$F \rightarrow B$ is a back edge.
Inner branch	$C \rightarrow F$	C, D, E	2 paths: $target = i \rightarrow F, \neg(target = i) \rightarrow F$	Loop-internal, but the selected region is acyclic.

TABLE V
BOUNDED NODE TREE FOR THE CFG-LEVEL INDEXED-FAMILY PREVIEW.

Depth	Node	Via	Next	Role	Detail
0	B	loop header	C / G	loop guard	branch on $i < n$
1	C	$i < n$	D / E	inner branch	branch on $target = i$
2	D / E	true / false arm	F	accumulator update	matches $+= 1$ or matches $+= 0$
3	F	latch	B	induction update	$i += 1$; back edge to the loop header

TABLE VI
CFG-LEVEL INDEXED-FAMILY COMPONENTS RECOVERED FROM THE MIR EXAMPLE.

Pattern	Evidence	Inferred sketch	Manual check
Loop guard	B	$i < n$	Matches the loop range guard.
Induction update	F	$i = + i \# 1; i := i + 1$	Linear constant-step update.
Guarded accumulator	D / E	matches $+= \text{ite}(target = k, 1, 0)$	Matches the per-iteration branch/update shape.
Indexed sketch	loop $B \leftarrow F$	for k in $[i0, n)$: matches $+= \text{ite}(target = k, 1, 0)$	Range shape matches the indexed summation in the text.

APPENDIX B
MANUAL QCE ORACLE FROM EFFICIENT STATE MERGING

This appendix manually applies the original QCE recurrence from Efficient State Merging to Listing 3 [3]. It uses a manually determined source-level dependence relation and produces the unhatted Q_t and Q_{add} oracle used to evaluate the automatic MIR approximations in Section III-C.

The source recurrence is

$$q(\ell, c) = \begin{cases} \beta q(\text{succ}(\ell), c) + \beta q(\ell', c) + c(\ell, e), & \text{instr}(\ell) = \text{if}(e) \text{ goto } \ell', \\ 0, & \text{instr}(\ell) = \text{halt}, \\ q(\text{succ}(\ell), c), & \text{otherwise.} \end{cases}$$

The two quantities used here are

$$Q_t(\ell) = q(\ell, \lambda(\ell', e).1)$$

and

$$Q_{\text{add}}(\ell_0, v) = q(\ell_0, \lambda(\ell', e).ite((\ell_0, v)\mathcal{C}(\ell', e), 1, 0)).$$

The relation $(\ell_0, v)\mathcal{C}(\ell', e)$ states that the value of variable v at location ℓ_0 may affect expression e evaluated at the later location ℓ' .

The CFG successor relation is: Entry $\rightarrow A$; A falls through to B and branches to C ; $B \rightarrow D$; $C \rightarrow D$; D falls through to E and branches to F ; $E \rightarrow G$; $F \rightarrow G$; $G \rightarrow \text{Exit}$; Exit is halt. The graph is acyclic, so no loop bound is needed.

There are two conditional expressions. For $e_A = (x < 0)$, the value of x at Entry and A may affect e_A . For $e_D = (t > 0)$, the source-level semantics before the branch is

$$t = ite(x < 0, 1 - x, x + 5) + y.$$

Therefore, the value of x and the value of y at any of Entry, A , B , C , and D may affect e_D . No later conditional expression is reachable from E , F , G , or Exit.

At the final conditional block,

$$Q_t(D) = 1, \quad Q_{\text{add}}(D, x) = 1, \quad Q_{\text{add}}(D, y) = 1.$$

Blocks B and C deterministically reach D , so the same values propagate without a β factor:

$$Q_t(B) = Q_t(C) = 1, \quad Q_{\text{add}}(B, x) = Q_{\text{add}}(C, x) = 1, \quad Q_{\text{add}}(B, y) = Q_{\text{add}}(C, y) = 1.$$

At block A , the first branch contributes one local query, and both successors contribute with factor β :

$$Q_t(A) = 1 + \beta(Q_t(B) + Q_t(C)) = 1 + 2\beta.$$

For x , the branch at A contributes locally and the later branch at D contributes through both successors:

$$Q_{\text{add}}(A, x) = 1 + \beta(Q_{\text{add}}(B, x) + Q_{\text{add}}(C, x)) = 1 + 2\beta.$$

For y , there is no local contribution at A , but the later condition at D depends on y :

$$Q_{\text{add}}(A, y) = \beta(Q_{\text{add}}(B, y) + Q_{\text{add}}(C, y)) = 2\beta.$$

Entry deterministically reaches A , so Entry has the same values as A . For E , F , G , and Exit, no future conditional expression remains: $Q_t = 0$ and $Q_{\text{add}}(\ell, x) = Q_{\text{add}}(\ell, y) = 0$. The resulting oracle values for $\beta \in \{0.50, 0.75, 0.90\}$ are reported in Table II(b), alongside the automatic MIR estimates in Table II(a).